

Cross-Traffic Intersection Management for Autonomous Vehicles

Embedded Electronic Engineering Lab A

Team A · Group 01

MD Azadul Islam · MD Jubair Salehin Razin · MD Rafiul Islam · Faysal Ahammed Tonmoy

Supervisors:
Prof. Dr. Stefan Henkler
Prof. Dr. Achim Rettberg

► Research Questions

How to model intersection traffic for autonomous cars using a queuing system?



What is the best scenario to reduce congestion at the intersection?



How to simulate with C-code (core logic) + freeRTOS, and which parameters matter?

► Motivation

The problem

- RSU grants crossing on demand — no fixed signal.
- Must stay provably safe: no collision, no deadlock.
- Must resolve priority and stay deterministic.

No signals

On-demand V2I grant, not fixed phases

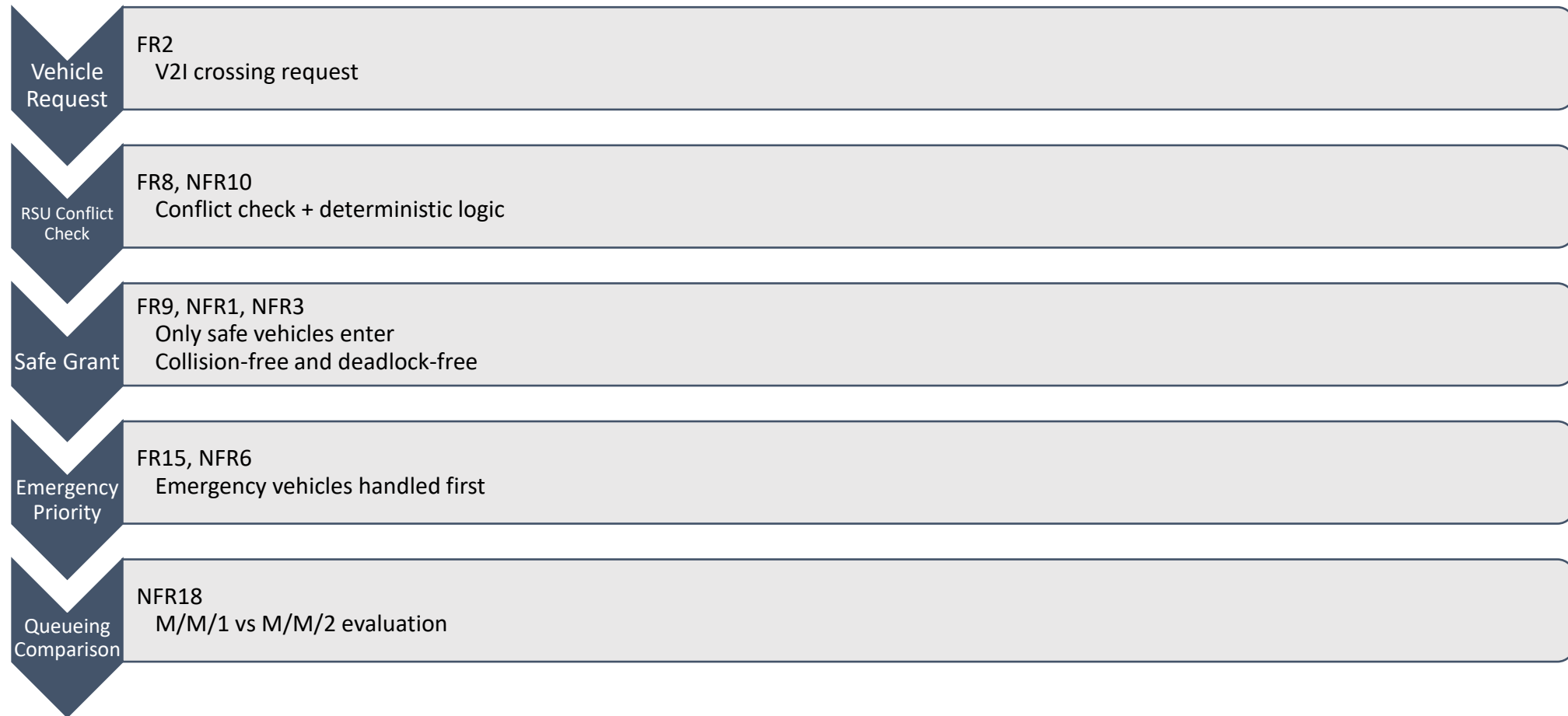
Provably safe

Collision- & deadlock-free, verified

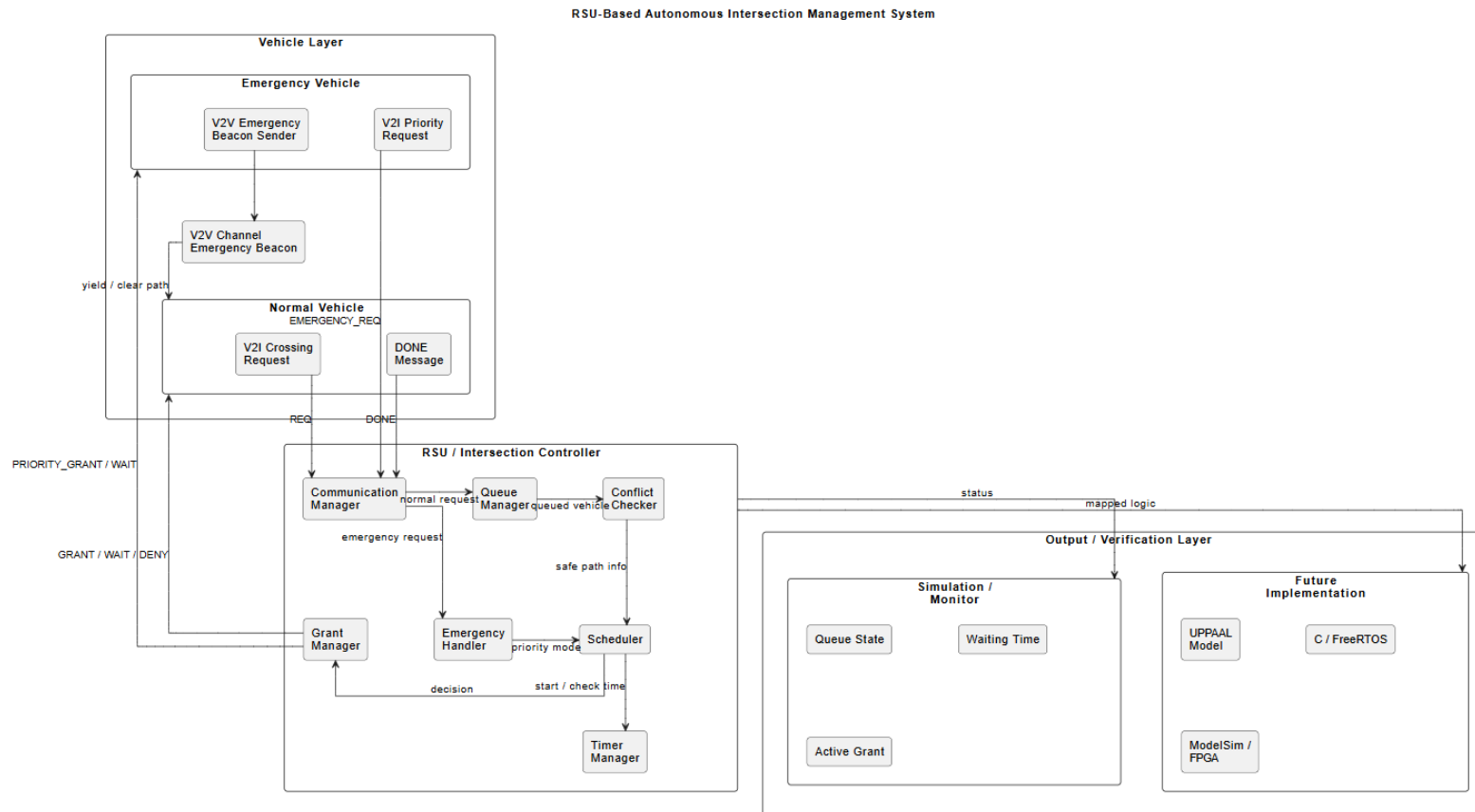
Emergency-aware

Ambulance pre-empts; safety kept

► Functional & non-functional requirements



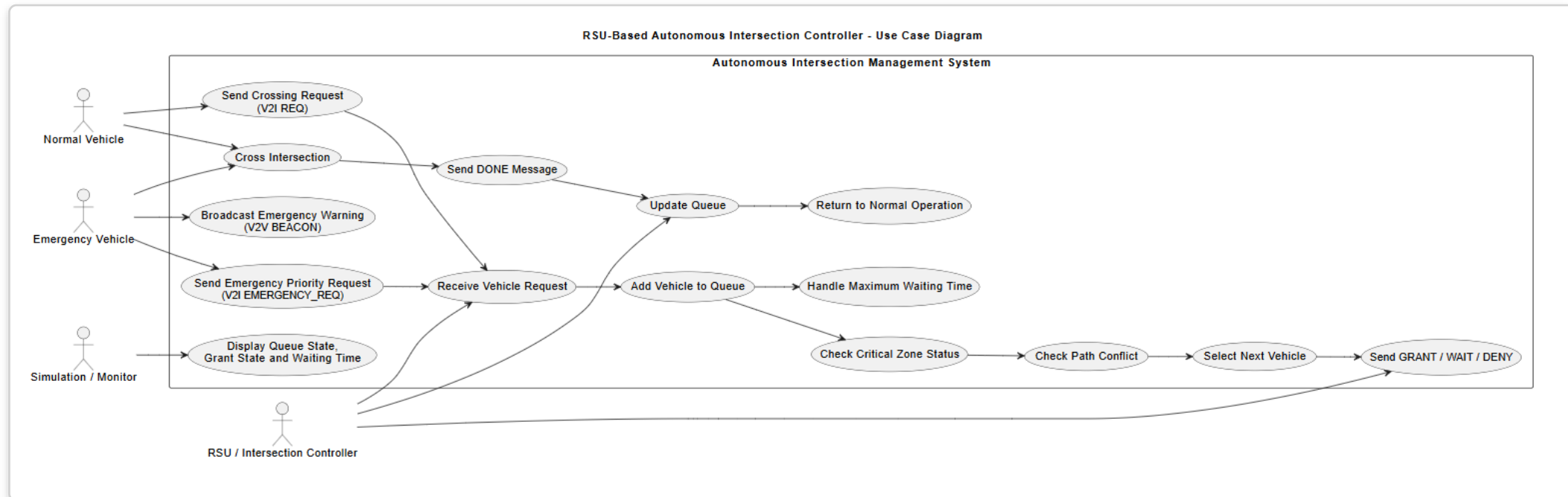
► Layered system architecture



Three layers

- Vehicles ↔ RSU (V2I / V2V).
- RSU: queue, conflict, grant.
- Shared state arrays.
- Coprocessor → safe_to_grant.

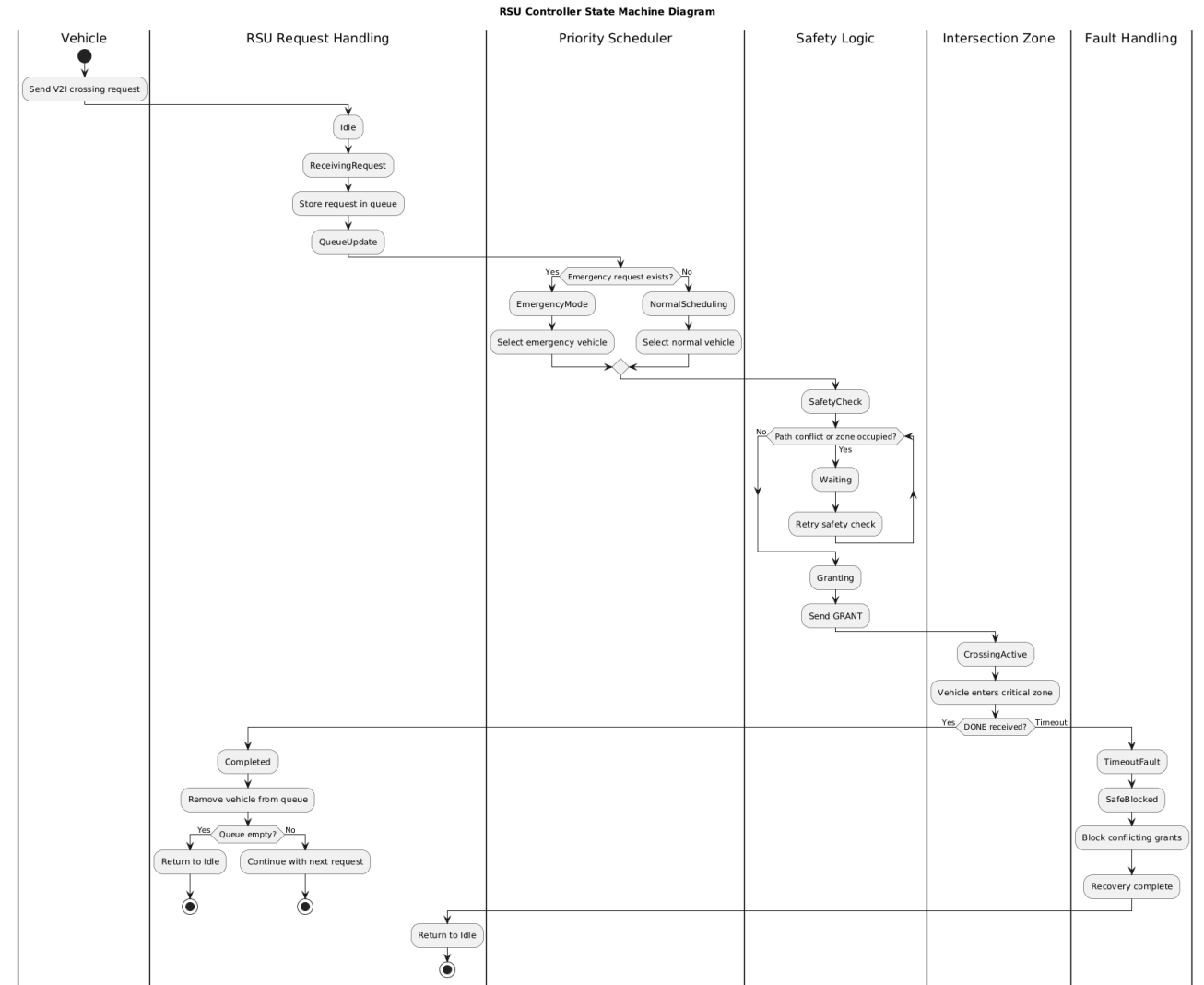
► Requirements — use-case diagram



Actors: Normal Vehicle · Emergency Vehicle · RSU / Intersection Controller · Simulation / Monitor

► State Machine Diagram

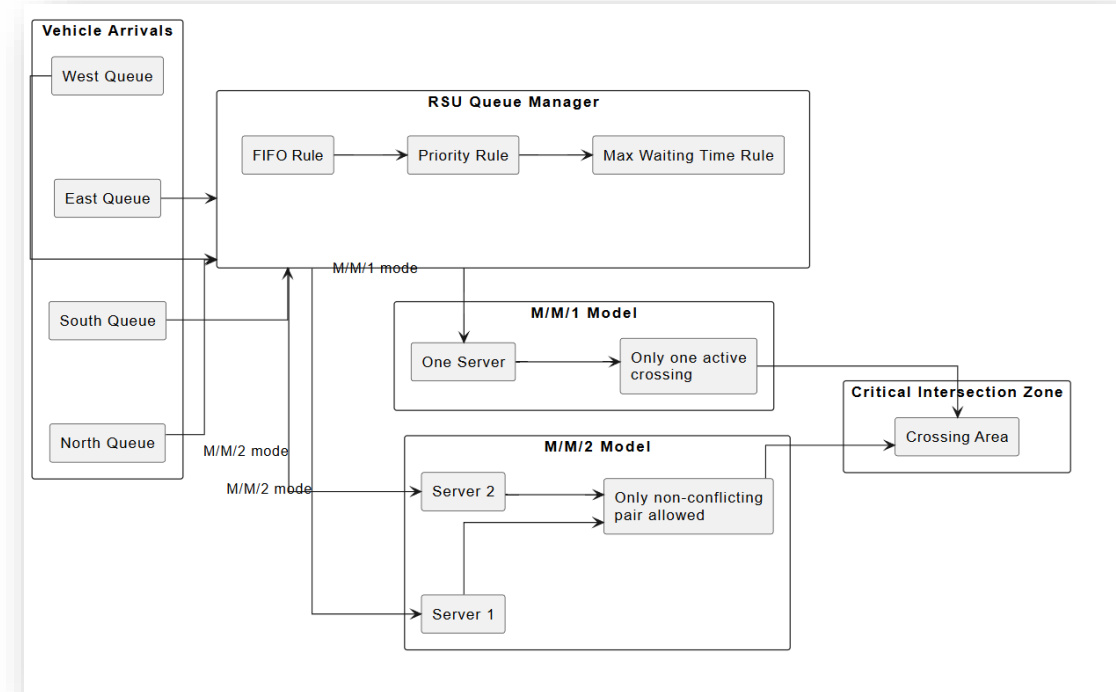
Two templates. Vehicle: waits → requests → grant → crosses → leaves. RSU: updates queues, checks priority & safety, grants.



► Six-vehicle intersection scenario

V	Origin	Move	Emg	Role
V0	East	Straight	Yes	Emergency, main street
V1	East	Straight	No	Main, M/M/2 partner
V2	West	Straight	No	Main, opposite dir.
V3	North	Straight	No	Side street
V4	North	Right	No	Critical right-turn
V5	South	Left	No	Critical left-turn

Sim parameters: $N = 6$ · MM_MODE $\in \{1, 2\}$ · period = {30,6,8,10,12,12} ·
crossTime = {4,5,5,5,5,5}



► One shared queue vs four approach queues

Single queue

- All vehicles in one shared queue.
- Grants the globally best safe vehicle.
- Simplest to verify first.
- Smaller state space.

Four queues

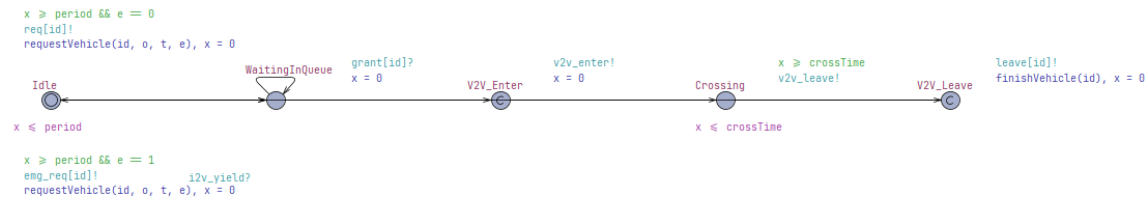
- $\text{queueLen}[R]$, $R = 4$ — one per road.
- Head-of-line: only each head eligible.
- RSU picks the best head of four.
- Like a real per-lane controller.

Shared either way: four queues feed one shared crossing — not four servers.

► Single-queue vs four-queue: what differs

Aspect	Single-queue model	Four-queue model
Queue structure	One shared priority queue	Four per-approach queues
Service	Global priority selection	Head-of-line + arbitration
Priority rules	Emg > main > side; right<left	Identical
Capacity	Shared; $\text{crossingCount} \leq \text{MODE}$	Identical, shared
Conflict handling	Intersection-wide	Identical
Properties	All hold	All hold + queue bounds
State space	Smaller, simpler	Slightly larger
Represents	One pooled resource	Per-lane controller

► UPPAAL Model



How it works

- Enqueue into queue[origin].
- headOfQueue(r) picks the head.
- canGrant() blocks non-heads.
- crossingCount ≤ MM_MODE.
- Queue bounds verified.

```

i : int[0,5]
req[i]?

i : int[0,5]
emg_req[i]?
emergencyNotified = false

emergencyWaiting() && !emergencyNotified
i2v_yield!
emergencyNotified = true

RSU_Control

```

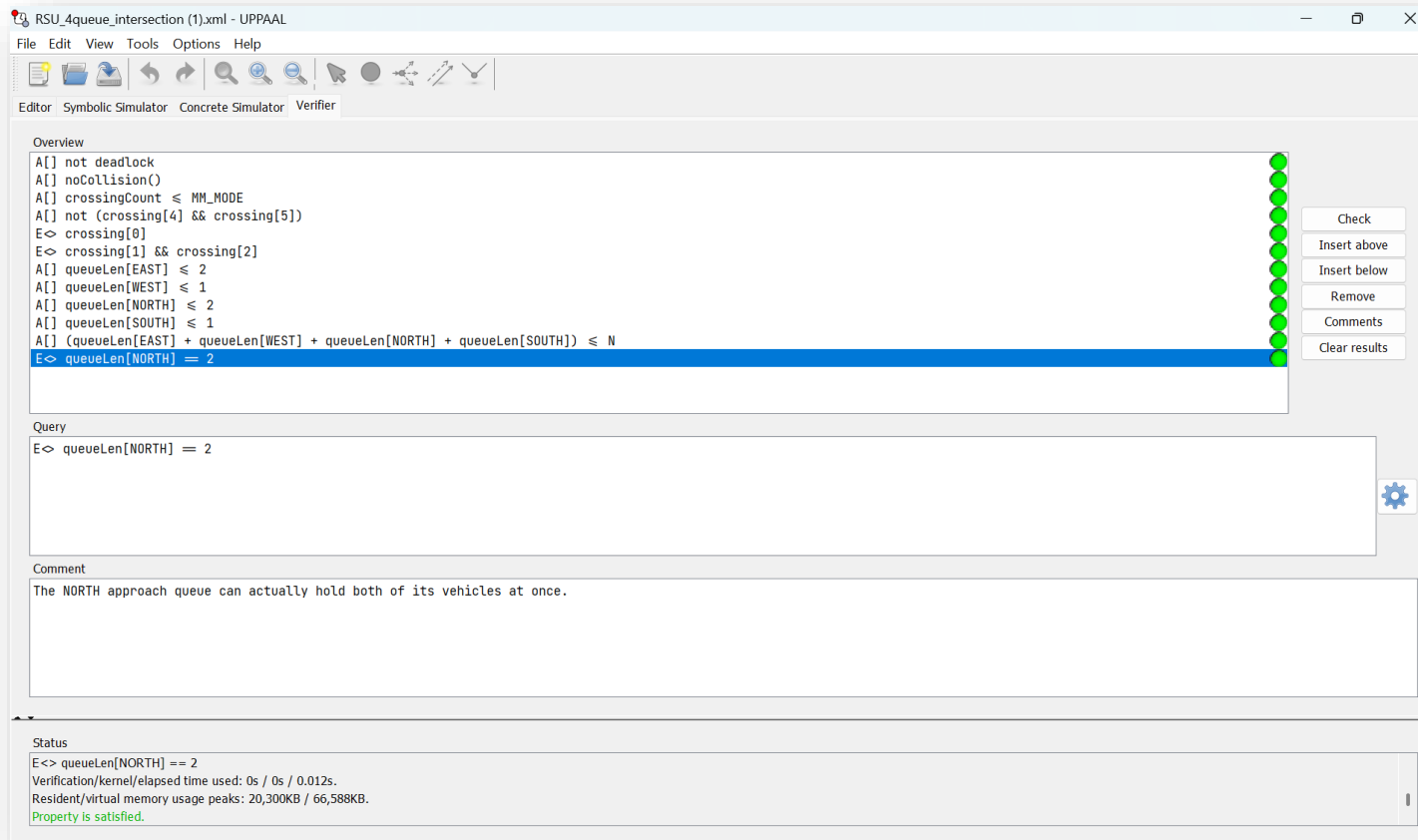
```

i : int[0,5]
canGrant(i)
grant[i]!
grantVehicle(i)

i : int[0,5]
leave[i]?

```

► Both models pass every safety query



✓ **A[] not deadlock**
Deadlock-free

✓ **A[] noCollision()**
No conflicting pair crosses

✓ **crossingCount ≤ MODE**
Capacity respected

✓ **not (cross[4] && cross[5])**
Critical V4/V5 exclusion

✓ **E<> crossing[0]**
Emergency can cross

✓ **E<> cross[1] && cross[2]**
M/M/2 overlap reachable

► M/M/2 removes waiting and shortens clearance

0.0

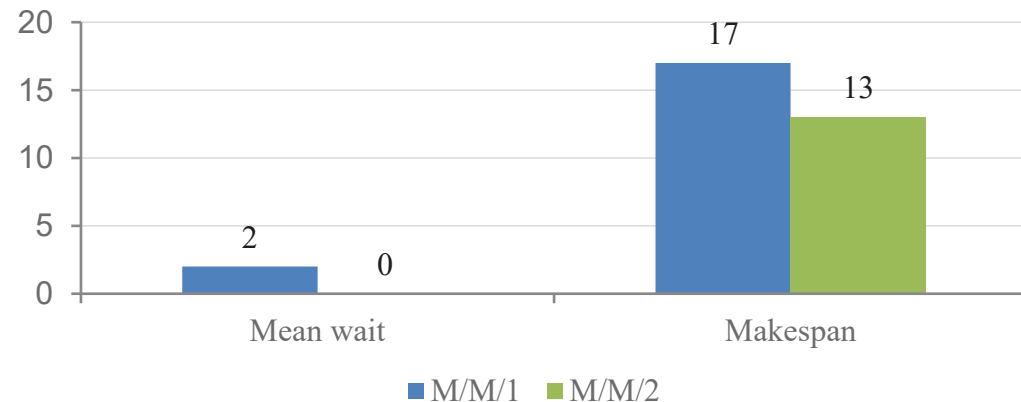
Mean wait (V1,V2)

was 2.0 in M/M/1

-24%

Time to clear pair

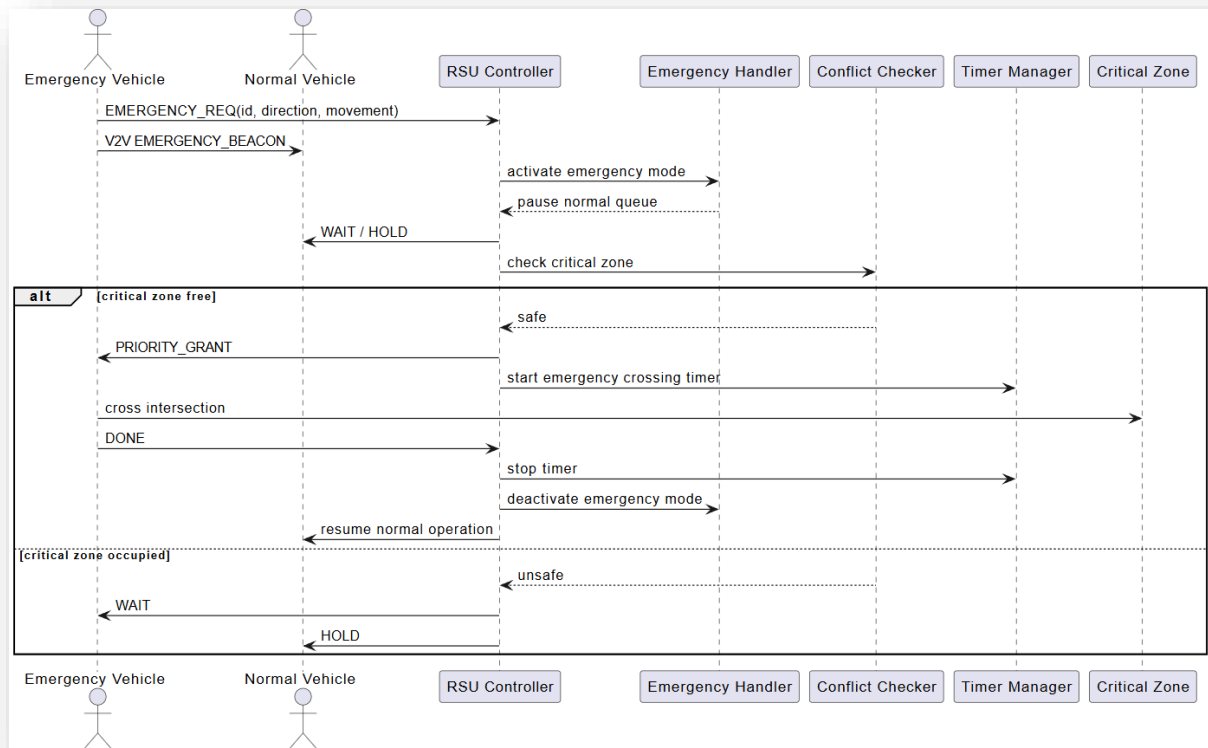
makespan 17 → 13



Metric	M/M/1	M/M/2
V1 waiting time	0	0
V2 waiting time	4	0
Mean wait (V1,V2)	2.0	0.0
Max concurrent	1	2
Makespan (pair)	17	13

Safety kept: V4 & V5 still serialised — the 2nd slot is only for non-conflicting moves.

► Pre-emption and the aging fix for V5



Emergency (V0)

- RSU blocks new grants + I2V yield.
- Granted only when safe.
- Changes order — not safety.

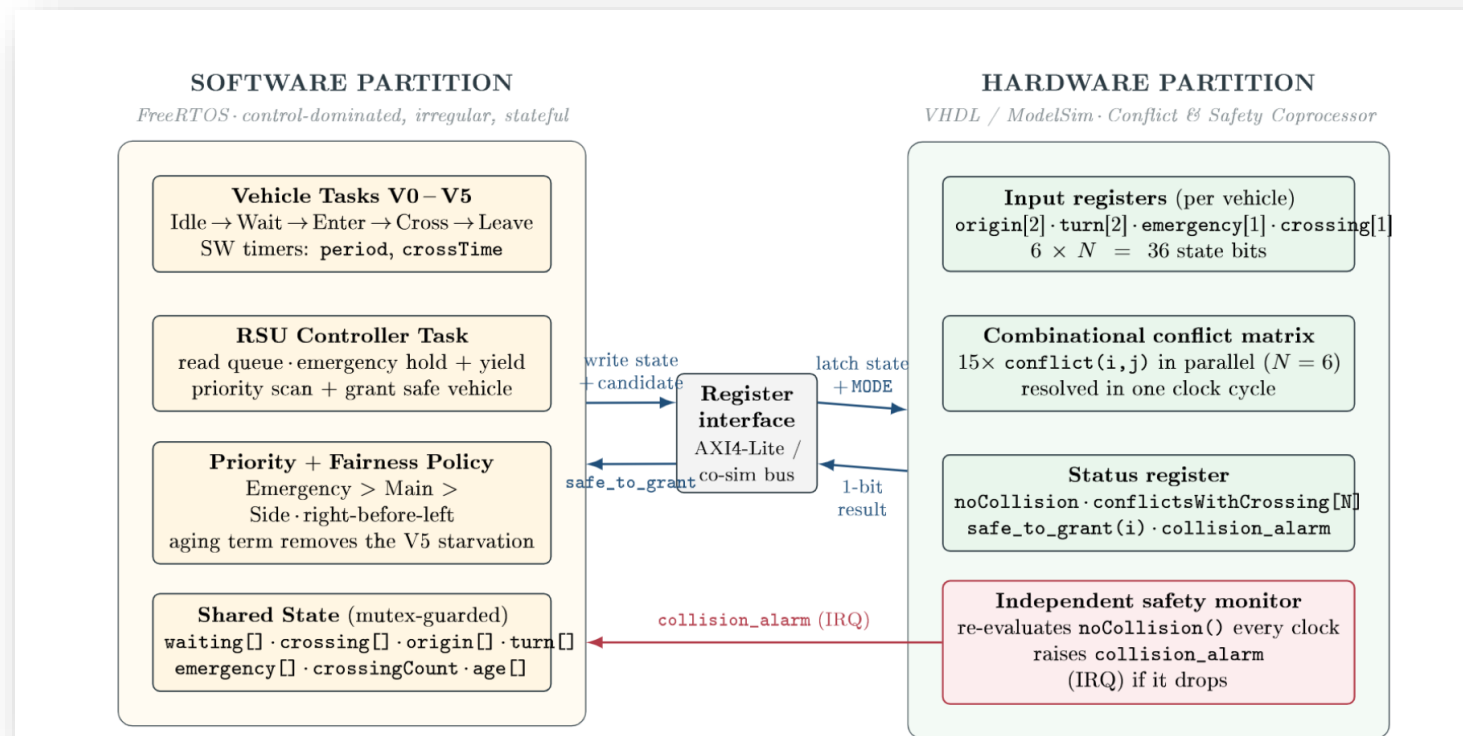
Fairness (aging) — fixes V5

Strict priority starves V5. Aging raises its effective priority over time:

$$\text{effPriority} = \text{priorityValue} - k \cdot \text{age}$$

Never overrides safety → bounded waiting.

► HW/SW codesign partition



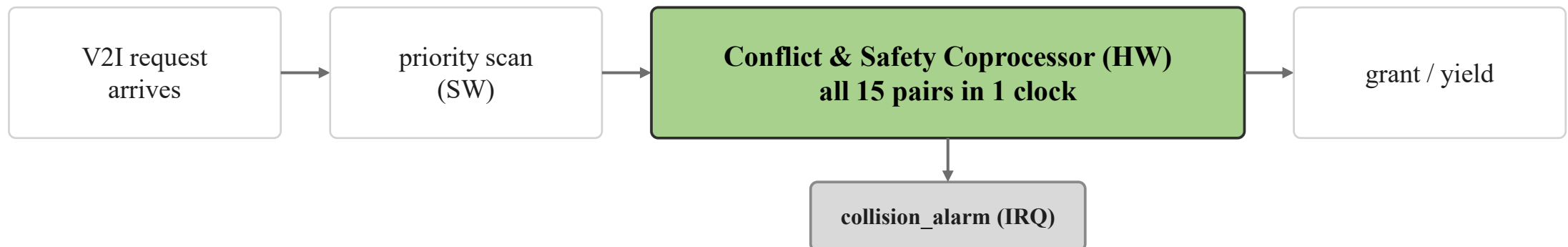
Criterion: regular · parallel · timing-critical → HARDWARE irregular · control-heavy → SOFTWARE

► One grant: $O(N^2)$ conflict work $\rightarrow$ one clock

software only (sequential)



HW/SW codesign (offloaded)



► A combinational safe-to-grant oracle

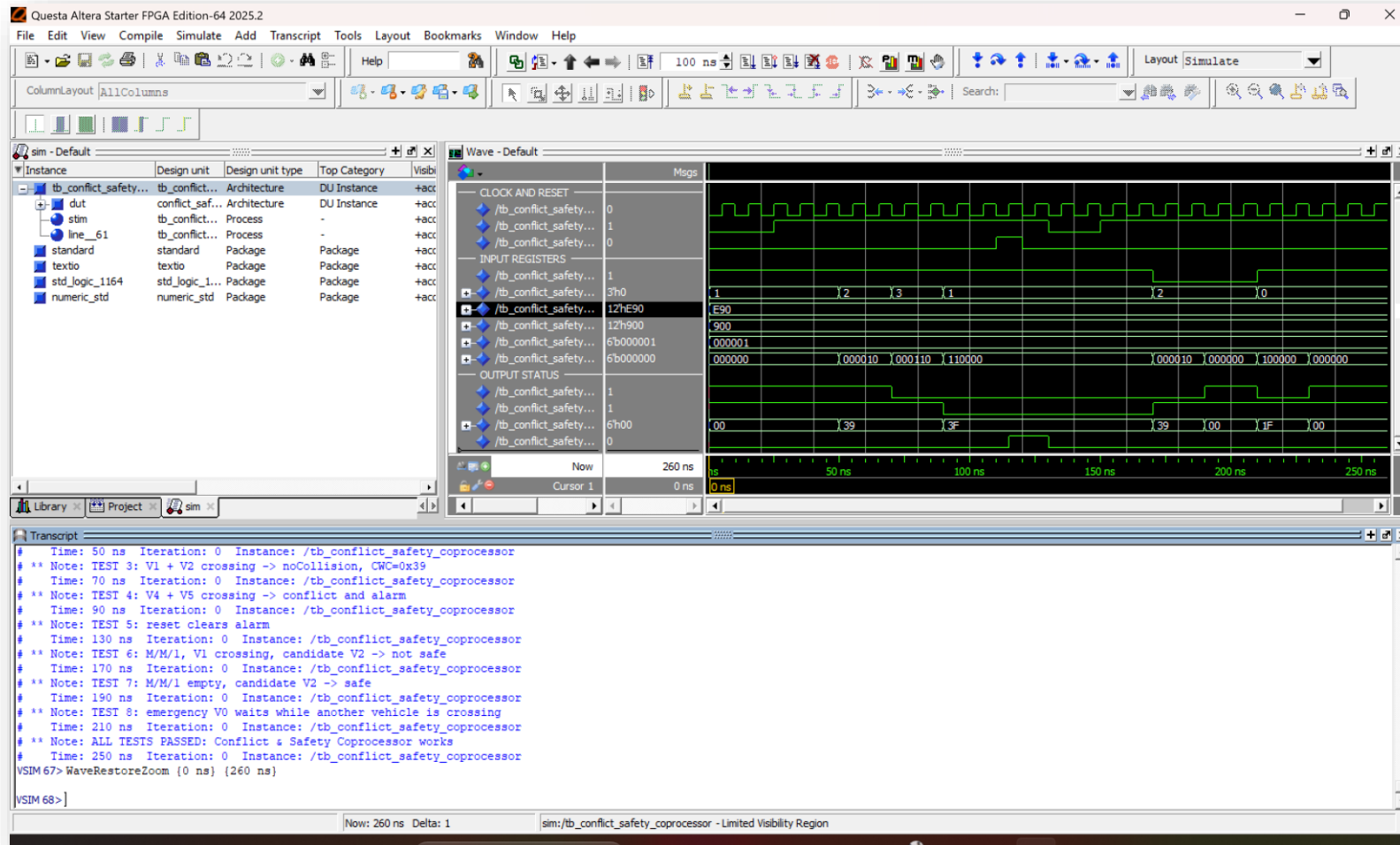
What it does

- Inputs: origin, turn, emergency, crossing.
- 15 conflict cells in parallel → 1 clock.
- Answers: is a candidate safe to grant?
- Monitor re-checks every clock.
- Raises collision_alarm (IRQ).

Intent: the safety decision is one bounded-latency bit
— write state, latch, read result.

Register	Meaning
STATE_VEH[i] W	origin, turn, emergency, crossing
MODE W	0 = M/M/1, 1 = M/M/2
CAND W	candidate vehicle id to test
ST_SAFE R	safe_to_grant(CAND)
ST_NOCOL R	noCollision() over fleet
ST_CWC R	conflictsWithCrossing vector
ST_ALARM R/IRQ	latched collision alarm

► VHDL coprocessor verified in QuestaSim

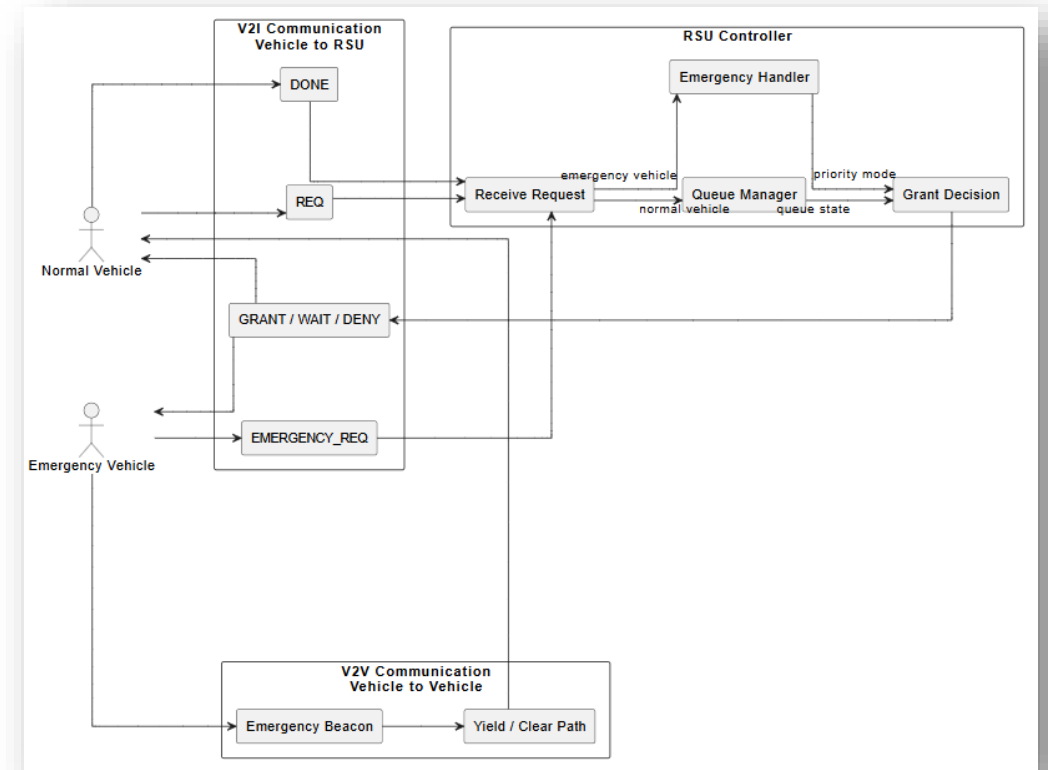


All 8 tests pass

- T3 V1+V2 → no collision
- T4 V4+V5 → conflict + alarm
- T6 M/M/1: V2 not safe
- T7 M/M/1 empty: V2 safe
- T8 emergency V0 waits

► FreeRTOS software: tasks, queues, protocol

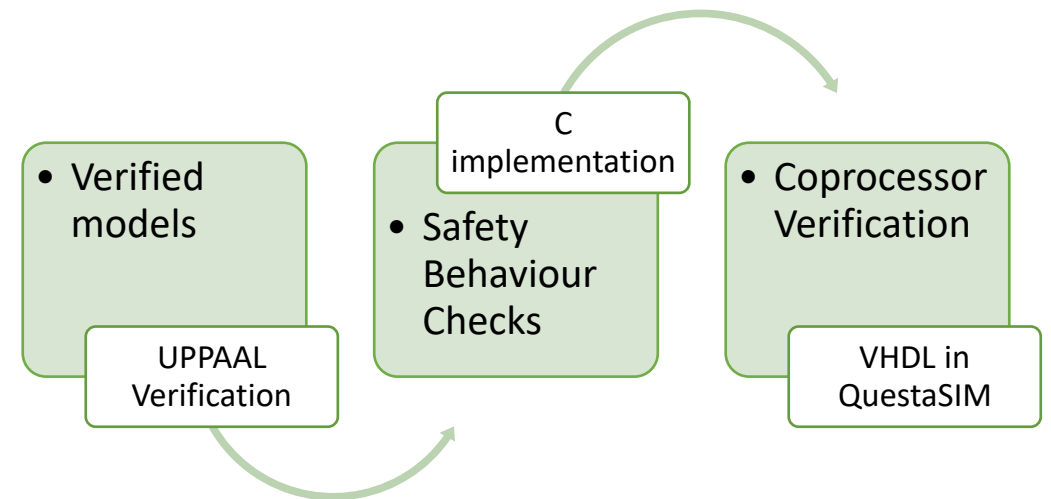
Component	Role
Vehicle task V0–V5	Request, wait, cross, leave
RSU task	Priority scan + grant
Request queue	V2I / emergency msgs
Grant queue	I2V grant messages
Leave queue	Completion messages
Yield broadcast	Emergency yield
Aging logic	Bounded waiting
hw_safe_to_grant	Register safety query



► Test & Validation

ID	Test case	Status
T1	M/M/1: crossingCount ≤ 1	Pass (C)
T2	M/M/2: V1 and V2 together	Pass (C)
T3	Emergency V0 highest priority	Pass (C)
T4	I2V yield broadcast appears	Pass (C)
T5	V4/V5 never together	C+UPPAAL+HW
T6	Fairness: V5 served (aging)	Pass (C)
T7	No collision-assertion fail	Pass (C)
T8	HW safe_to_grant / alarm	QuestaSim

Chain:



► Limitations & future work

Current limits

- Model + C + VHDL verified.
- FreeRTOS ↔ coprocessor co-sim to do.
- Numbers from one scenario.
- Periodic arrivals: really D/D/c.

Next steps

- Co-simulate RSU ↔ coprocessor.
- End-to-end timing on real stack.
- Wait-time vs arrival-rate study.
- Synthesise & deploy to FPGA.

► A verified RSU controller, realised in HW/SW

1. Formally safe

Deadlock-free, collision-free, capacity-bounded — proven in UPPAAL.

2. Right queue model

Single- and four-queue variants equivalent; four add per-lane realism.

3. Quantified benefit

M/M/2 removes the wait and clears ~24% faster — safety kept.

4. Codesign realised

Conflict check as a VHDL coprocessor, verified; control in FreeRTOS.

Thank you

Questions & discussion
